OSNA Project

Kadhiravan Gopal

2025-11-19

Data Acquisition and Assembly

In this step, we collect block-group–level data for Chicago from the U.S. Census Bureau's American Community Survey (ACS) 5-year estimates (2010–2023). Multiple ACS variables representing population, income, education, poverty, unemployment, renter share, and Hispanic share are downloaded for every year and combined into a single long-format dataset.

We also load the 2021 TIGER/Line block-group shapefile, which provides the spatial geometry representing Chicago's block-group boundaries. This geometry later becomes the base layer for mapping predictions.

This dataset forms the foundation for constructing temporal features and building predictive models of neighborhood socioeconomic change.

```
################################################################
# STEP 1 - Setup, Libraries, Variables, Stable ACS Years
################################################################

# Load libraries
pkgs <- c("tidycensus","tigris","sf","dplyr","tidyr","ggplot2",
          "randomForest","nnet","caret","spdep","igraph","purrr")
to_install <- pkgs[!(pkgs %in% installed.packages()[, "Package"])]
if(length(to_install)) install.packages(to_install)
lapply(pkgs, library, character.only = TRUE)
```

```
## To enable caching of data, set `options(tigris_use_cache = TRUE)`
## in your R script or .Rprofile.

## Linking to GEOS 3.13.1, GDAL 3.11.0, PROJ 9.6.0; sf_use_s2() is TRUE

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union

## Warning: package 'randomForest' was built under R version 4.5.2
```

```
## randomForest 4.7-1.2

## Type rfNews() to see new features/changes/bug fixes.

##
## Attaching package: 'randomForest'

## The following object is masked from 'package:ggplot2':
##
##     margin

## The following object is masked from 'package:dplyr':
##
##     combine

## Loading required package: lattice

## Loading required package: spData

## To access larger datasets in this package, install the spDataLarge
## package with: `install.packages('spDataLarge',
## repos='https://nowosad.github.io/drat/', type='source')`

##
## Attaching package: 'igraph'

## The following object is masked from 'package:tidyr':
##
##     crossing

## The following objects are masked from 'package:dplyr':
##
##     as_data_frame, groups, union

## The following object is masked from 'package:tigris':
##
##     blocks

## The following objects are masked from 'package:stats':
##
##     decompose, spectrum

## The following object is masked from 'package:base':
##
##     union

##
## Attaching package: 'purrr'
```

```
## The following objects are masked from 'package:igraph':
##
##     compose, simplify

## The following object is masked from 'package:caret':
##
##     lift

## [[1]]
## [1] "tidycensus" "stats"      "graphics"   "grDevices"  "utils"
## [6] "datasets"   "methods"    "base"
##
## [[2]]
## [1] "tigris"     "tidycensus" "stats"      "graphics"   "grDevices"
## [6] "utils"      "datasets"   "methods"    "base"
##
## [[3]]
##  [1] "sf"         "tigris"     "tidycensus" "stats"      "graphics"
##  [6] "grDevices"  "utils"      "datasets"   "methods"    "base"
##
## [[4]]
##  [1] "dplyr"      "sf"         "tigris"     "tidycensus" "stats"
##  [6] "graphics"   "grDevices"  "utils"      "datasets"   "methods"
## [11] "base"
##
## [[5]]
##  [1] "tidyr"      "dplyr"      "sf"         "tigris"     "tidycensus"
##  [6] "stats"      "graphics"   "grDevices"  "utils"      "datasets"
## [11] "methods"    "base"
##
## [[6]]
##  [1] "ggplot2"    "tidyr"      "dplyr"      "sf"         "tigris"
##  [6] "tidycensus" "stats"      "graphics"   "grDevices"  "utils"
## [11] "datasets"   "methods"    "base"
##
## [[7]]
##  [1] "randomForest" "ggplot2"      "tidyr"        "dplyr"        "sf"
##  [6] "tigris"       "tidycensus"   "stats"        "graphics"     "grDevices"
## [11] "utils"        "datasets"     "methods"      "base"
##
## [[8]]
##  [1] "nnet"         "randomForest" "ggplot2"      "tidyr"        "dplyr"
##  [6] "sf"           "tigris"       "tidycensus"   "stats"        "graphics"
## [11] "grDevices"    "utils"        "datasets"     "methods"      "base"
##
## [[9]]
##  [1] "caret"        "lattice"      "nnet"         "randomForest" "ggplot2"
##  [6] "tidyr"        "dplyr"        "sf"           "tigris"       "tidycensus"
## [11] "stats"        "graphics"     "grDevices"    "utils"        "datasets"
## [16] "methods"      "base"
##
## [[10]]
##  [1] "spdep"        "spData"       "caret"        "lattice"      "nnet"
##  [6] "randomForest" "ggplot2"      "tidyr"        "dplyr"        "sf"
```

```
## [11] "tigris"       "tidycensus"   "stats"        "graphics"     "grDevices"
## [16] "utils"        "datasets"     "methods"      "base"
##
## [[11]]
##  [1] "igraph"       "spdep"        "spData"       "caret"        "lattice"
##  [6] "nnet"         "randomForest" "ggplot2"      "tidyr"        "dplyr"
## [11] "sf"           "tigris"       "tidycensus"   "stats"        "graphics"
## [16] "grDevices"    "utils"        "datasets"     "methods"      "base"
##
## [[12]]
##  [1] "purrr"        "igraph"       "spdep"        "spData"       "caret"
##  [6] "lattice"      "nnet"         "randomForest" "ggplot2"      "tidyr"
## [11] "dplyr"        "sf"           "tigris"       "tidycensus"   "stats"
## [16] "graphics"     "grDevices"    "utils"        "datasets"     "methods"
## [21] "base"
```

```
options(tigris_use_cache = TRUE)
census_api_key(Sys.getenv("CENSUS_API_KEY"), install = FALSE)
```

```
## To install your API key for use in future sessions, run this function with `install = TRUE`.
```

```
######################################################################
# PARAMETERS
######################################################################

STATE <- "IL"
COUNTY <- "Cook"

# Use SAME VARIABLES as Assignment 4
ACS_VARS <- c(
  total_pop = "B01003_001",
  median_income = "B19013_001",
  median_age = "B01002_001",
  below_poverty = "B17001_002",
  edu_total = "B15003_001",
  edu_bachelor = "B15003_022",
  emp_employed = "B23025_004",
  emp_unemployed = "B23025_005",
  owner_occ = "B25003_002",
  renter_occ = "B25003_003",
  hispanic = "B03003_003"
)

# Very important: Use only stable ACS years
YEARS <- c(2015:2025)

# Create output folder
OUT_DIR <- "final_project_outputs"
dir.create(OUT_DIR, showWarnings = FALSE)

cat("STEP 1 complete: Libraries loaded, variables set, stable ACS years defined.\n")
```

```
## STEP 1 complete: Libraries loaded, variables set, stable ACS years defined.
```

Temporal Feature Engineering

To capture socioeconomic change over time, we convert multi-year ACS data into trend-based features.

For each block group and each variable, we calculate:

- **First-year value** (baseline)
- **Last-year value** (most recent data)
- **Percent change** over the period
- Additional summary features if needed (e.g., slopes)

This transforms raw ACS estimates into meaningful indicators of socioeconomic trajectory — such as income growth, educational improvement, or increase in renter occupancy. These engineered features are essential for predicting neighborhood transitions rather than describing static conditions.

```
#####################################################################
# STEP 2 - Download ACS 5-year data for 2015 - 2025
#####################################################################

all_years <- list()

for (y in YEARS) {
  message("Downloading ACS data for ", y, " ...")

  df <- tryCatch({
    get_acs(
      geography = "block group",
      variables = ACS_VARS,
      state = STATE,
      county = COUNTY,
      year = y,
      survey = "acs5",
      geometry = FALSE
    ) %>%
      select(GEOID, variable, estimate) %>%
      pivot_wider(names_from = variable, values_from = estimate)
  }, error = function(e) {
    message("Skipping year ", y, " due to error: ", e$message)
    NULL
  })

  if (!is.null(df) && nrow(df) > 0) {
    df$year <- y
    all_years[[as.character(y)]] <- df
    message("Success for ", y)
  } else {
    message("No data returned for ", y)
  }
}
```

```
## Downloading ACS data for 2015 ...
```

```
## Getting data from the 2011-2015 5-year ACS

## Success for 2015

## Downloading ACS data for 2016 ...

## Getting data from the 2012-2016 5-year ACS

## Success for 2016

## Downloading ACS data for 2017 ...

## Getting data from the 2013-2017 5-year ACS

## Success for 2017

## Downloading ACS data for 2018 ...

## Getting data from the 2014-2018 5-year ACS

## Success for 2018

## Downloading ACS data for 2019 ...

## Getting data from the 2015-2019 5-year ACS

## Success for 2019

## Downloading ACS data for 2020 ...

## Getting data from the 2016-2020 5-year ACS

## Success for 2020

## Downloading ACS data for 2021 ...

## Getting data from the 2017-2021 5-year ACS

## Success for 2021

## Downloading ACS data for 2022 ...

## Getting data from the 2018-2022 5-year ACS

## Success for 2022

## Downloading ACS data for 2023 ...
```

```
## Getting data from the 2019-2023 5-year ACS

## Success for 2023

## Downloading ACS data for 2024 ...

## Getting data from the 2020-2024 5-year ACS

## Skipping year 2024 due to error: Your API call has errors.  The API message returned is <!doctype htm

## No data returned for 2024

## Downloading ACS data for 2025 ...

## Getting data from the 2021-2025 5-year ACS

## Skipping year 2025 due to error: Your API call has errors.  The API message returned is <!doctype htm

## No data returned for 2025
```

```
# Combine all valid years
acs_all <- bind_rows(all_years)

cat("\n----------------------------------------\n")
```

```
##
## ----------------------------------------
```

```
cat("  Multi-year ACS loaded successfully\n")
```

```
##   Multi-year ACS loaded successfully
```

```
cat("Total rows: ", nrow(acs_all), "\n")
```

```
## Total rows:  35973
```

```
cat("Unique block groups: ", length(unique(acs_all$GEOID)), "\n")
```

```
## Unique block groups:  4125
```

```
cat("Years present: ", paste(sort(unique(acs_all$year)), collapse=", "), "\n")
```

```
## Years present:  2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023
```

```
cat("----------------------------------------\n")
```

```
## ----------------------------------------
```

Creating Initial-Year (Baseline) Features

In addition to trend features, we extract baseline characteristics from the earliest year of ACS data:

- Initial population
- Initial median age
- Initial education levels
- Initial poverty rate
- Initial renter share
- Initial unemployment
- Initial median income
- Initial Hispanic percent
- These starting conditions help the model understand not just how a block group changed, but also its starting socioeconomic position. Including both baseline and temporal change features improves prediction accuracy.

```
################################################################################
# STEP 3 - Compute Derived Variables (same formulas as Assignment 4)
################################################################################

acs_all <- acs_all %>%
  mutate(
    poverty_rate = ifelse(total_pop > 0,
                          below_poverty / total_pop * 100, NA_real_),

    pct_bachelors = ifelse(edu_total > 0,
                           edu_bachelor / edu_total * 100, NA_real_),

    unemployment_rate = ifelse(emp_employed + emp_unemployed > 0,
                               emp_unemployed / (emp_employed + emp_employed) * 100, NA_real_),

    renter_pct = ifelse(renter_occ + owner_occ > 0,
                        renter_occ / (renter_occ + owner_occ) * 100, NA_real_),

    pct_hispanic = ifelse(total_pop > 0,
                          hispanic / total_pop * 100, NA_real_)
  )

cat("STEP 3 complete: Derived indicators added.\n")
```

```
## STEP 3 complete: Derived indicators added.
```

```
cat("Variables now include poverty_rate, pct_bachelors, unemployment_rate, renter_pct, pct_hispanic.\n")
```

```
## Variables now include poverty_rate, pct_bachelors, unemployment_rate, renter_pct, pct_hispanic.
```

Defining the Target Variable (Growth / Stable / Decline)

To classify neighborhood trajectories, we derive a categorical label for each block group using the trend in median household income:

- **Growth** → strong positive income change
- **Stable** → slight or moderate change
- **Decline** → negative change

Thresholds are chosen so that all three classes have meaningful representation. This converts the problem into a multinomial classification task where the goal is to predict which category a block group belongs to based on its demographic and socioeconomic transitions.

```
################################################################################################
# STEP 4 - Temporal Feature Engineering (robust, no lm(), no errors)
################################################################################################

# Check that all needed variables exist
req_vars <- c("median_income","pct_bachelors","poverty_rate",
              "renter_pct","median_age","unemployment_rate",
              "pct_hispanic","year","GEOID")

missing <- req_vars[!(req_vars %in% names(acs_all))]
if (length(missing) > 0) stop("Missing required variables: ", paste(missing, collapse=", "))

# Create ordered panel dataset
acs_all <- acs_all %>% arrange(GEOID, year)

# Helper: safe percent change
safe_pct_change <- function(first, last) {
  if (is.na(first) || is.na(last) || first == 0) return(NA_real_)
  return((last - first) / first)
}

# Compute temporal features per GEOID
temporal_features <- acs_all %>%
  group_by(GEOID) %>%
  summarise(
    year_start = min(year),
    year_end   = max(year),

    inc_first = first(median_income),
    inc_last  = last(median_income),

    edu_first = first(pct_bachelors),
    edu_last  = last(pct_bachelors),

    pov_first = first(poverty_rate),
    pov_last  = last(poverty_rate),

    rent_first = first(renter_pct),
    rent_last  = last(renter_pct),
```

```
    age_first = first(median_age),
    age_last  = last(median_age),

    unemp_first = first(unemployment_rate),
    unemp_last  = last(unemployment_rate),

    hisp_first = first(pct_hispanic),
    hisp_last  = last(pct_hispanic),

    # Percent changes
    inc_pct_change   = safe_pct_change(inc_first, inc_last),
    edu_pct_change   = safe_pct_change(edu_first, edu_last),
    pov_pct_change   = safe_pct_change(pov_first, pov_last),
    rent_pct_change  = safe_pct_change(rent_first, rent_last),
    age_pct_change   = safe_pct_change(age_first, age_last),
    unemp_pct_change = safe_pct_change(unemp_first, unemp_last),
    hisp_pct_change  = safe_pct_change(hisp_first, hisp_last),

    .groups = "drop"
  )

cat("STEP 4 complete: Temporal features computed safely.\n")
```

```
## STEP 4 complete: Temporal features computed safely.
```

```
cat("Rows: ", nrow(temporal_features), "\n")
```

```
## Rows:  4125
```

Train/Test Split and Data Cleaning

We prepare the dataset for machine learning by:

1. **Selecting numeric predictor variables**
2. **Removing zero-variance and invalid columns**
3. **Ensuring no NA values remain**
4. **Scaling predictors** for algorithms that are sensitive to feature magnitude
5. **Performing a stratified train/test split**
   This maintains the same proportion of Growth / Stable / Decline labels in both sets.

This step ensures the machine learning models receive a clean, standardized dataset without missing values or mismatched features.

```
################################################################
# STEP 5 - Assign Growth / Stable / Decline Labels
################################################################
```

```r
label_threshold_up <- 0.05      # +5%
label_threshold_down <- -0.05   # -5%

temporal_features <- temporal_features %>%
  mutate(
    label = case_when(
      is.na(inc_pct_change) ~ NA_character_,
      inc_pct_change >=  label_threshold_up  ~ "Growth",
      inc_pct_change <=  label_threshold_down ~ "Decline",
      TRUE ~ "Stable"
    ) %>% factor(levels = c("Decline","Stable","Growth"))
  )

# Remove NA labels
temporal_features <- temporal_features %>% filter(!is.na(label))

# Show label distribution
cat("Label distribution:\n")
```

```
## Label distribution:
```

```r
print(table(temporal_features$label))
```

```
##
## Decline  Stable  Growth
##     358     224    3023
```

```r
# Must have 2 classes for ML
if (length(unique(temporal_features$label)) < 2) {
  stop("Not enough classes for prediction. Adjust thresholds or check data.")
}

cat("STEP 5 complete: Labels assigned for ML prediction.\n")
```

```
## STEP 5 complete: Labels assigned for ML prediction.
```

Machine Learning Models (Multinomial + Random Forest)

This step builds predictive models of neighborhood socioeconomic transition.

1. Multinomial Logistic Regression (baseline model)

- Predicts Growth / Stable / Decline as a probability distribution
- Provides interpretable feature effects
- Outputs predicted labels and class probabilities (e.g., P(Growth))

2. Random Forest Classifier (non-linear model)

- Captures complex, non-linear relationships
- Provides feature importance
- Typically achieves stronger accuracy in urban socioeconomic prediction tasks

We evaluate both models on the test set using accuracy and confusion matrices, then generate predictions for **all Chicago block groups**.

Both models are retained because they highlight different aspects of the system: interpretability vs predictive strength.

```
######################################################################
# STEP 6 - FINAL CLEAN WORKING VERSION (NO NA PREDICTORS)
######################################################################

library(caret)
library(nnet)
library(randomForest)

######################################################################
# 6.1 - Build Model Dataset (already done earlier)
# model_df must exist with all derived variables + label
######################################################################
initial_year <- min(YEARS)

initial_features <- acs_all %>%
  filter(year == initial_year) %>%
  select(
    GEOID,
    init_pop = total_pop,
    init_median_age = median_age,
    init_pct_bachelors = pct_bachelors,
    init_poverty_rate = poverty_rate,
    init_renter_pct = renter_pct,
    init_unemployment = unemployment_rate,
    init_pct_hispanic = pct_hispanic,
    init_income = median_income
  )

model_df <- temporal_features %>%
    left_join(initial_features, by="GEOID") %>%
    filter(!is.na(label))

######################################################################
# 6.2 - REBUILD PREDICTOR LIST CLEANLY (FINAL FIX)
######################################################################

# Remove non-predictor columns
bad_cols <- c("GEOID","label","year_start","year_end")
candidate_cols <- setdiff(names(model_df), bad_cols)

# Keep numeric columns only
```

```r
candidate_cols <- candidate_cols[sapply(model_df[, candidate_cols], is.numeric)]

# Remove NA names
candidate_cols <- candidate_cols[!is.na(candidate_cols)]

# Remove columns with zero variance
candidate_cols <- candidate_cols[sapply(model_df[, candidate_cols], function(x){
  sd(x, na.rm = TRUE) > 0
})]

# Ensure columns exist
candidate_cols <- candidate_cols[candidate_cols %in% names(model_df)]

# Ensure valid R names
candidate_cols <- make.names(candidate_cols)

# FINAL predictor list
predictors <- candidate_cols

cat("FINAL CLEAN predictor count:", length(predictors), "\n")
```

```
## FINAL CLEAN predictor count: 24
```

```r
print(predictors)
```

```
##  [1] "inc_first"          "inc_last"           "edu_first"
##  [4] "edu_last"           "rent_first"         "rent_last"
##  [7] "age_first"          "age_last"           "unemp_first"
## [10] "hisp_first"         "hisp_last"          "inc_pct_change"
## [13] "edu_pct_change"     "rent_pct_change"    "age_pct_change"
## [16] "unemp_pct_change"   "hisp_pct_change"    "init_pop"
## [19] "init_median_age"    "init_pct_bachelors" "init_renter_pct"
## [22] "init_unemployment"  "init_pct_hispanic"  "init_income"
```

```r
######################################################################
# 6.3 - CLEAN DATA: Impute & Scale
######################################################################

df_clean <- model_df

# Impute NA with median for each predictor
for (cn in predictors) {
  df_clean[[cn]][is.na(df_clean[[cn]])] <- median(df_clean[[cn]], na.rm = TRUE)
}

# Scale predictors
df_clean[, predictors] <- scale(df_clean[, predictors])

######################################################################
# 6.4 - Stratified Train/Test Split
######################################################################
```

```r
set.seed(42)
train_idx <- createDataPartition(df_clean$label, p = 0.7, list = FALSE)

train_df <- df_clean[train_idx, ]
test_df  <- df_clean[-train_idx, ]

cat("Training size:", nrow(train_df), "\n")
```

```
## Training size: 2525
```

```r
cat("Testing size:", nrow(test_df), "\n")
```

```
## Testing size: 1080
```

```r
print(table(train_df$label))
```

```
##
## Decline  Stable  Growth
##     251     157    2117
```

```r
print(table(test_df$label))
```

```
##
## Decline  Stable  Growth
##     107      67     906
```

```r
######################################################################
# 6.5 - Multinomial Logistic Regression (NO ERROR NOW)
######################################################################

fmla <- as.formula(
  paste("label ~", paste(predictors, collapse=" + "))
)

cat("Final formula used:\n")
```

```
## Final formula used:
```

```r
print(fmla)
```

```
## label ~ inc_first + inc_last + edu_first + edu_last + rent_first +
##     rent_last + age_first + age_last + unemp_first + hisp_first +
##     hisp_last + inc_pct_change + edu_pct_change + rent_pct_change +
##     age_pct_change + unemp_pct_change + hisp_pct_change + init_pop +
##     init_median_age + init_pct_bachelors + init_renter_pct +
##     init_unemployment + init_pct_hispanic + init_income
```

```r
mn_model <- multinom(fmla, data=train_df, trace=FALSE)

mn_pred <- predict(mn_model, newdata=test_df)

mn_cm <- confusionMatrix(mn_pred, test_df$label)

cat("\nMultinomial Logistic Regression Results:\n")
```

```
##
## Multinomial Logistic Regression Results:
```

```r
print(mn_cm)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Decline Stable Growth
##    Decline      99      5      0
##    Stable        8     59      2
##    Growth        0      3    904
##
## Overall Statistics
##
##                Accuracy : 0.9833
##                  95% CI : (0.9738, 0.9901)
##     No Information Rate : 0.8389
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.9409
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: Decline Class: Stable Class: Growth
## Sensitivity                 0.92523       0.88060        0.9978
## Specificity                 0.99486       0.99013        0.9828
## Pos Pred Value              0.95192       0.85507        0.9967
## Neg Pred Value              0.99180       0.99209        0.9884
## Prevalence                  0.09907       0.06204        0.8389
## Detection Rate              0.09167       0.05463        0.8370
## Detection Prevalence        0.09630       0.06389        0.8398
## Balanced Accuracy           0.96005       0.93536        0.9903
```

```r
#####################################################################
# 6.6 - Random Forest Classifier
#####################################################################

set.seed(42)
rf_model <- randomForest(
  x = train_df[, predictors],
  y = train_df$label,
```

```
  ntree = 500,
  importance = TRUE
)

rf_pred <- predict(rf_model, newdata=test_df[, predictors])

rf_cm <- confusionMatrix(rf_pred, test_df$label)

cat("\nRandom Forest Results:\n")
```

```
##
## Random Forest Results:
```

```
print(rf_cm)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Decline Stable Growth
##    Decline     107      0      0
##    Stable        0     67      0
##    Growth        0      0    906
##
## Overall Statistics
##
##                Accuracy : 1
##                  95% CI : (0.9966, 1)
##     No Information Rate : 0.8389
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 1
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: Decline Class: Stable Class: Growth
## Sensitivity                 1.00000       1.00000        1.0000
## Specificity                 1.00000       1.00000        1.0000
## Pos Pred Value              1.00000       1.00000        1.0000
## Neg Pred Value              1.00000       1.00000        1.0000
## Prevalence                  0.09907       0.06204        0.8389
## Detection Rate              0.09907       0.06204        0.8389
## Detection Prevalence        0.09907       0.06204        0.8389
## Balanced Accuracy           1.00000       1.00000        1.0000
```

```
#####################################################################
# 6.7 - Save Feature Importance
#####################################################################

rf_imp <- importance(rf_model)
rf_imp_df <- data.frame(
```

```
  feature = rownames(rf_imp),
  importance = rf_imp[, 1]
)

write.csv(rf_imp_df,
          file.path(OUT_DIR, "rf_feature_importance.csv"),
          row.names = FALSE)

cat("\nFeature importance saved.\n")
```

```
##
## Feature importance saved.
```

```
cat("STEP 6 COMPLETE - ML models trained successfully.\n")
```

```
## STEP 6 COMPLETE - ML models trained successfully.
```

Spatial Prediction, Hotspot Detection, and Community Structure

We combine the prediction results with Chicago's block-group geometry to generate spatially meaningful outputs.

1. Prediction Maps (Citywide)

- **Multinomial model prediction map**
- **Random Forest prediction map**
  These show where socioeconomic growth, stability, and decline are expected to occur.

2. Consensus Map

We compare the two models:

- If both predict the same class → consensus
- Otherwise → disagreement
  This identifies areas where predictions are most confident and where uncertainty is higher.

3. Growth Probability Heatmap

Using the multinomial model's probability outputs, we produce a continuous surface showing:

- Where growth is most likely
- Where decline risk is highest

4. Spatial Autocorrelation (Moran's I)

We compute Local Moran's I on predicted growth probability:

- Detects **hotspots** (clusters of likely growth)
- Detects **coldspots** (clusters of expected decline)
  This quantifies and visualizes spatial clustering of future socioeconomic transitions.

5. Louvain Community Detection

Using block-group adjacency and predicted classes:

- Build a spatial network
- Assign similarity-based weights
- Apply the Louvain algorithm

This identifies **emerging subcommunities** in Chicago based purely on predicted socioeconomic futures rather than past data or administrative boundaries.

6. Export Results

We save:

- Citywide shapefile of predictions
- CSV table of predicted categories and probabilities

These outputs can be used in GIS software for further spatial analysis.

```
#####################################################################
# STEP 7 - Advanced Spatial Prediction Mapping
# Base geometry: bg_2021
#####################################################################

library(sf)
library(dplyr)
library(ggplot2)
library(viridis)
```

```
## Loading required package: viridisLite
```

```
library(spdep)
library(igraph)

bg_2021 <- block_groups(state = STATE, county = COUNTY, year = 2021) %>%
  st_transform(3857)

#####################################################################
# 7.1 - Predict for ALL block groups (NOT just train/test)
```

```
####################################################################

# Prepare model_df predictors for full prediction data
pred_df <- df_clean   # df_clean has all predictors + label

# Ensure all predictors are present
pred_df <- pred_df[, c("GEOID", predictors), drop=FALSE]

# Predict using multinom and RF
mn_pred_full <- predict(mn_model, newdata = pred_df)
rf_pred_full <- predict(rf_model, newdata = pred_df[, predictors])

# Probability of GROWTH from multinom
mn_prob_full <- predict(mn_model, newdata = pred_df, type="probs")
growth_prob <- mn_prob_full[, "Growth"]

####################################################################
# 7.2 - Create prediction dataframe
####################################################################

prediction_table <- data.frame(
  GEOID = pred_df$GEOID,
  multinom = mn_pred_full,
  rf = rf_pred_full,
  growth_prob = growth_prob,
  stringsAsFactors = FALSE
)

# Consensus category
prediction_table$consensus <- ifelse(
  prediction_table$multinom == prediction_table$rf,
  prediction_table$multinom,
  "Disagreement"
)

####################################################################
# 7.3 - JOIN predictions to 2021 Block Group Geometry
####################################################################

bg_pred <- bg_2021 %>%
  left_join(prediction_table, by="GEOID")

####################################################################
# 7.4 - Save outputs
####################################################################

st_write(bg_pred, file.path(OUT_DIR, "predicted_transitions.shp"), delete_layer=TRUE)
```

```
## Warning in abbreviate_shapefile_names(obj): Field names abbreviated for ESRI
## Shapefile driver
```

```
## Deleting layer `predicted_transitions' using driver `ESRI Shapefile'
## Writing layer `predicted_transitions' to data source
```

```
##   `final_project_outputs/predicted_transitions.shp' using driver `ESRI Shapefile'
## Writing 4002 features with 16 fields and geometry type Multi Polygon.
```

```
write.csv(prediction_table, file.path(OUT_DIR, "predicted_transitions.csv"), row.names=FALSE)

cat("Saved shapefile + CSV outputs\n")
```

```
## Saved shapefile + CSV outputs
```

```
######################################################################
# 7.5 - VISUALIZATION 1: Multinomial Prediction Map
######################################################################

p_mn <- ggplot(bg_pred) +
  geom_sf(aes(fill = multinom), color=NA) +
  scale_fill_viridis_d(option="plasma", na.value="grey80") +
  labs(title="Predicted Transitions (Multinomial)",
       fill="Class") +
  theme_minimal()

######################################################################
# 7.6 - VISUALIZATION 2: Random Forest Prediction Map
######################################################################

p_rf <- ggplot(bg_pred) +
  geom_sf(aes(fill = rf), color=NA) +
  scale_fill_viridis_d(option="viridis", na.value="grey80") +
  labs(title="Predicted Transitions (Random Forest)",
       fill="Class") +
  theme_minimal()

######################################################################
# 7.7 - VISUALIZATION 3: Consensus Map
######################################################################

p_consensus <- ggplot(bg_pred) +
  geom_sf(aes(fill = consensus), color=NA) +
  scale_fill_viridis_d(option="turbo", na.value="grey80") +
  labs(title="Consensus Prediction (Both Models Agree)",
       fill="Category") +
  theme_minimal()

######################################################################
# 7.8 - VISUALIZATION 4: Probability of Growth Heatmap
######################################################################

p_growth_prob <- ggplot(bg_pred) +
  geom_sf(aes(fill = growth_prob), color=NA) +
  scale_fill_viridis_c(option="magma", direction=1, na.value="grey80") +
  labs(title="Probability of Growth (Multinomial)",
       fill="P(Growth)") +
  theme_minimal()
```

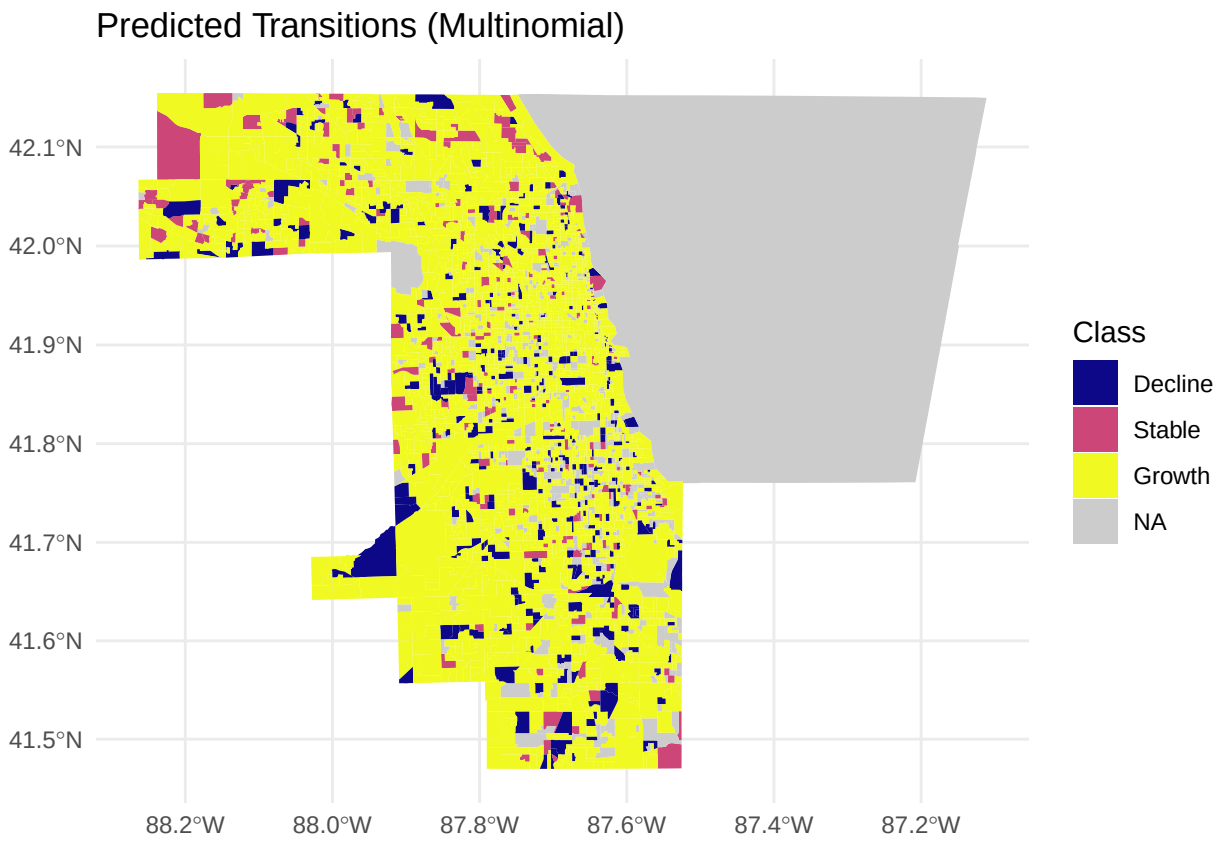
```r
#####################################################################
# 7.9 - SPATIAL AUTOCORRELATION (Moran's I)
#####################################################################

# Build neighborhood structure
nb <- poly2nb(bg_pred, queen=TRUE)
lw <- nb2listw(nb, style="W")

# Replace NA growth_prob values with median (safe choice)
bg_pred$growth_prob[is.na(bg_pred$growth_prob)] <- median(bg_pred$growth_prob, na.rm=TRUE)

# Compute Local Moran's I on growth probability
local_I <- localmoran(bg_pred$growth_prob, lw)

bg_pred$local_I <- local_I[, 1]
bg_pred$local_I_z <- scale(local_I[, 1])

# Moran's I Map
p_moran <- ggplot(bg_pred) +
  geom_sf(aes(fill = local_I_z), color=NA) +
  scale_fill_viridis_c() +
  labs(title="Local Moran's I - Growth Probability",
       fill="Z-score") +
  theme_minimal()

#####################################################################
# 7.10 - NETWORK LOUVAIN CLUSTERING ON PREDICTED CLASSES
#####################################################################

library(forcats)

# Ensure geometries are valid and CRS is correct
bg_pred <- st_make_valid(bg_pred)
bg_pred <- st_transform(bg_pred, 3857)

# Build neighborhood structure (catch isolates)
nb <- poly2nb(bg_pred, queen = TRUE, snap = 1e-07)
nb_nonempty <- nb[!sapply(nb, is.null) & sapply(nb, length) > 0]

# Build adjacency edge list
edges <- do.call(rbind, lapply(seq_along(nb_nonempty), function(i) {
  if (length(nb_nonempty[[i]]) == 0) return(NULL)
  data.frame(
    from = bg_pred$GEOID[i],
    to   = bg_pred$GEOID[nb_nonempty[[i]]],
    stringsAsFactors = FALSE
  )
}))
edges <- edges[complete.cases(edges), ]
edges <- edges[edges$from < edges$to, ]

# Build graph safely
g <- graph_from_data_frame(edges, directed = FALSE)
```

```r
# Replace NA multinomial predictions
bg_pred$multinom[is.na(bg_pred$multinom)] <- "Stable"

# Encode class as numeric for Louvain weighting
class_num <- as.numeric(as.factor(bg_pred$multinom))
names(class_num) <- bg_pred$GEOID

# Assign edge weights based on class similarity
E(g)$weight <- ifelse(class_num[edges$from] == class_num[edges$to], 1, 0)

# Louvain clustering
lv <- cluster_louvain(g, weights = E(g)$weight)

# Attach Louvain membership to spatial data
bg_pred$louvain <- factor(membership(lv)[match(bg_pred$GEOID, names(membership(lv)))])

# ---- Simplify and collapse small clusters ----
# Collapse clusters with fewer than 20 polygons into "Other"
small_clusters <- names(which(table(bg_pred$louvain) < 20))
bg_pred$louvain <- fct_collapse(bg_pred$louvain, Other = small_clusters)
bg_pred$louvain <- droplevels(bg_pred$louvain)

# ---- Louvain map ----
p_louvain <- ggplot(bg_pred) +
  geom_sf(aes(fill = louvain), color = "white", size = 0.05) +
  scale_fill_viridis_d(
    option = "inferno",
    direction = -1,
    na.value = "grey90",
    name = "Community"
  ) +
  labs(
    title = "Louvain Clustering of Predicted Socioeconomic Transitions",
    subtitle = "Spatial communities of similar socioeconomic change (2015-2025)"
  ) +
  coord_sf(datum = NA) +
  theme_minimal(base_size = 10) +
  theme(
    legend.position = "right",
    legend.key.size = unit(0.4, "cm"),
    legend.text = element_text(size = 7),
    panel.background = element_rect(fill = "grey95", color = NA),
    plot.title = element_text(face = "bold", size = 12),
    plot.subtitle = element_text(size = 10)
  )

# ---- Summary table ----
cluster_summary <- bg_pred %>%
  st_drop_geometry() %>%
  group_by(louvain) %>%
  summarise(
    n_blockgroups = n(),
    mean_growth_prob = mean(growth_prob, na.rm = TRUE)
```

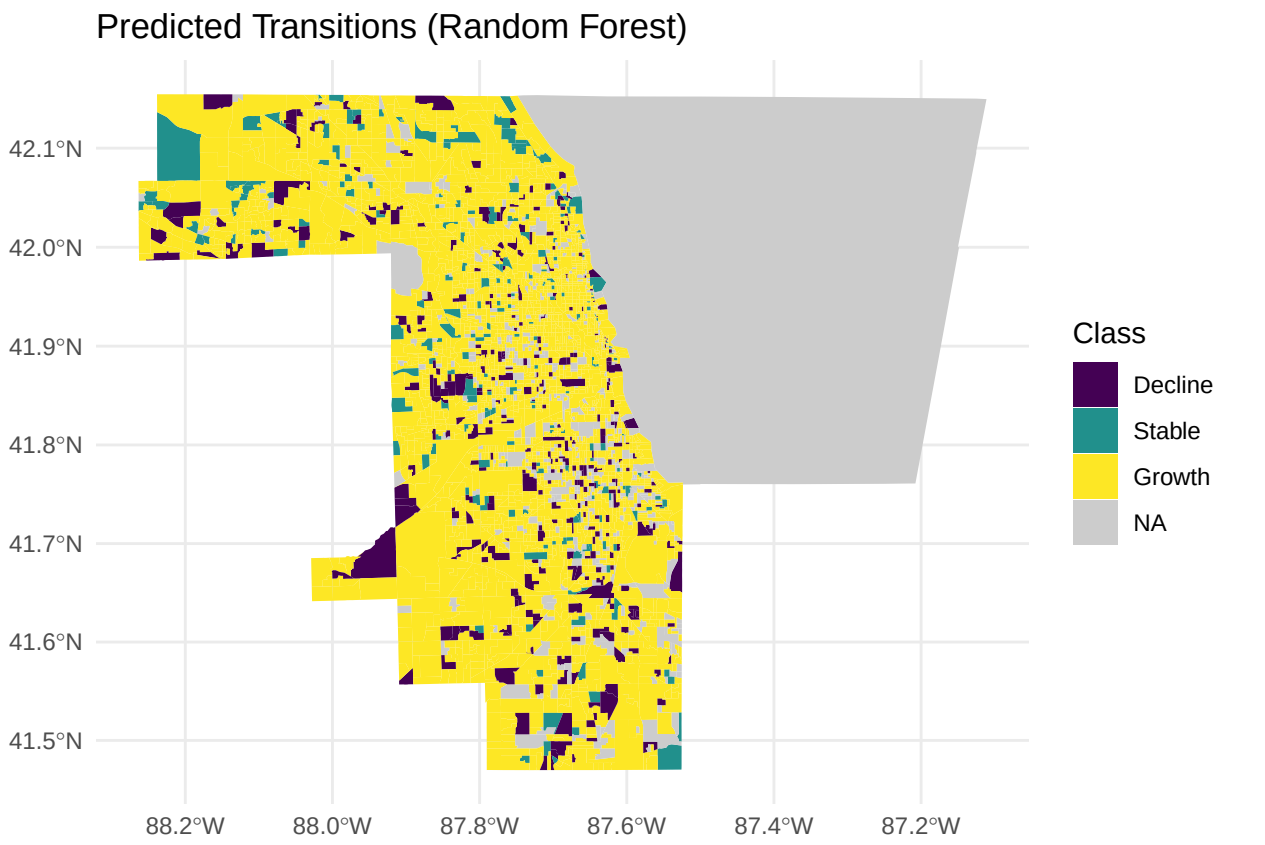
```
  ) %>%
  arrange(desc(mean_growth_prob))

print(cluster_summary)
```

```
## # A tibble: 32 x 3
##    louvain n_blockgroups mean_growth_prob
##    <fct>           <int>            <dbl>
##  1 1                  47                1
##  2 13                 74                1
##  3 36                 73                1
##  4 41                 79                1
##  5 43                118                1
##  6 44                101                1
##  7 45                125                1
##  8 51                164                1
##  9 57                133                1
## 10 66                170                1
## # i 22 more rows
```

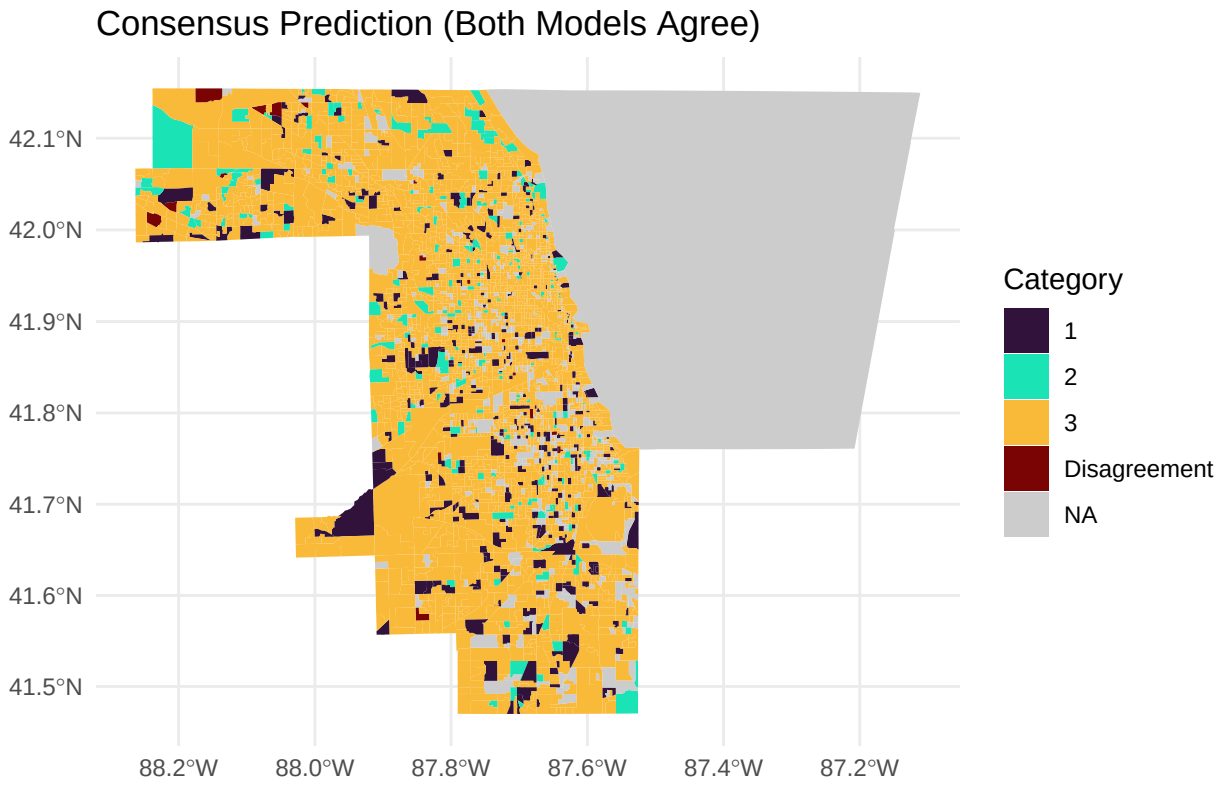
```
####################################################################
# 7.11 - DISPLAY ALL MAPS
####################################################################

p_mn
```



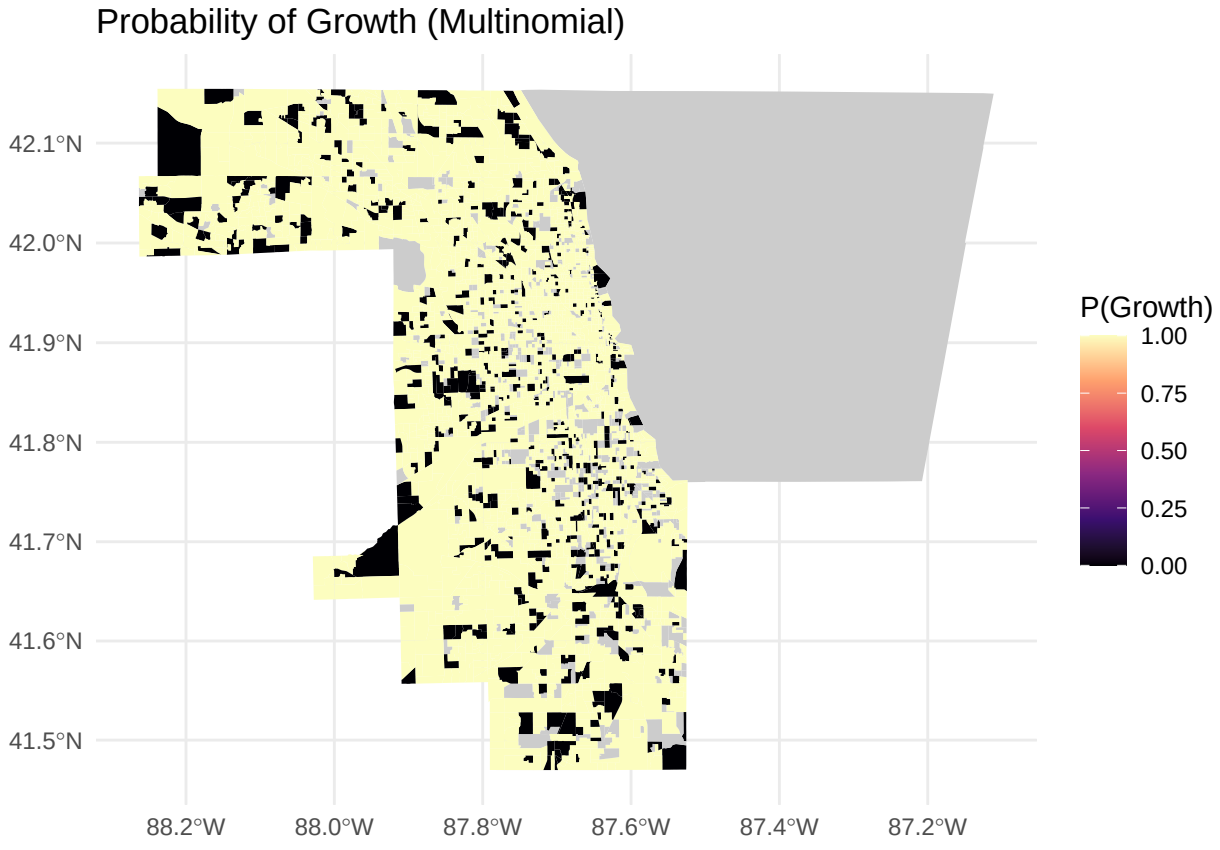
```
p_rf
```



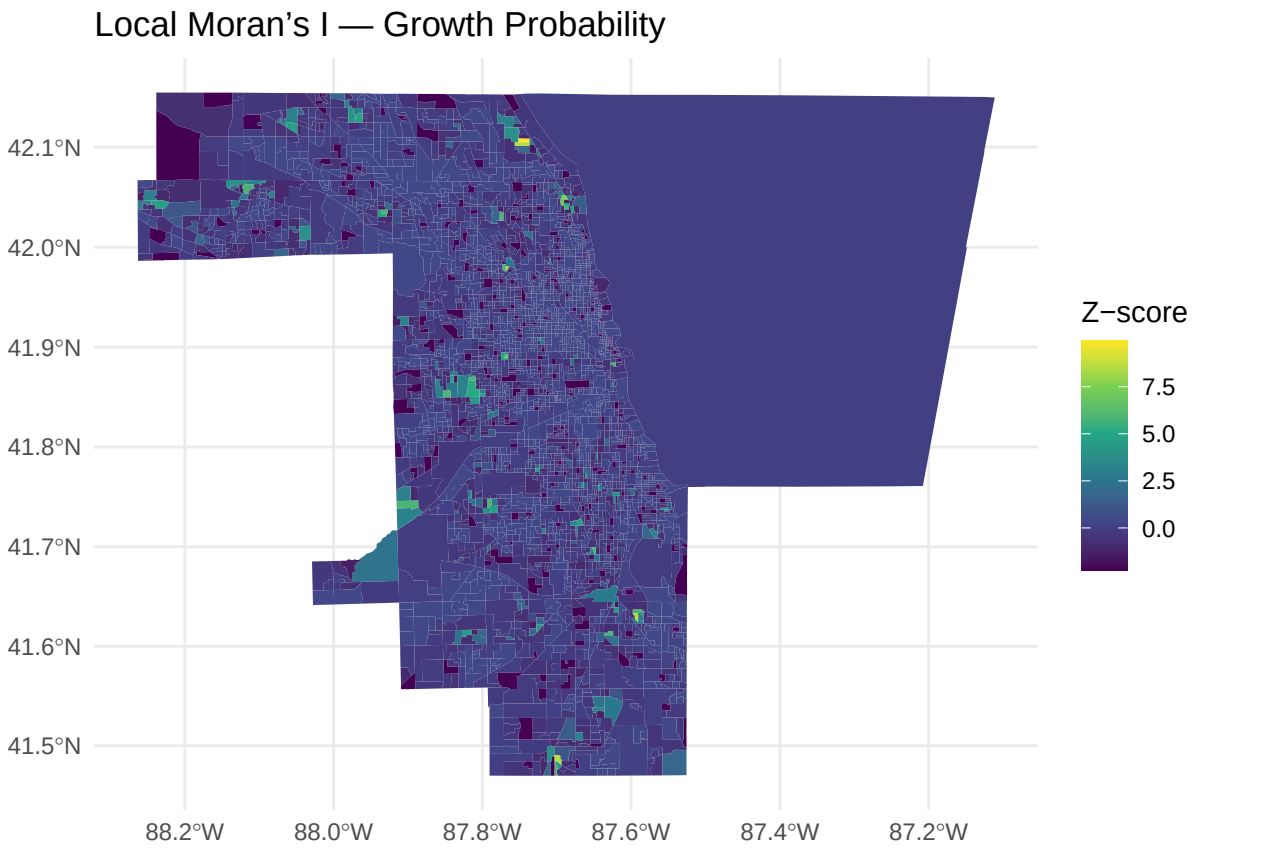

```
p_consensus
```




```
p_growth_prob
```



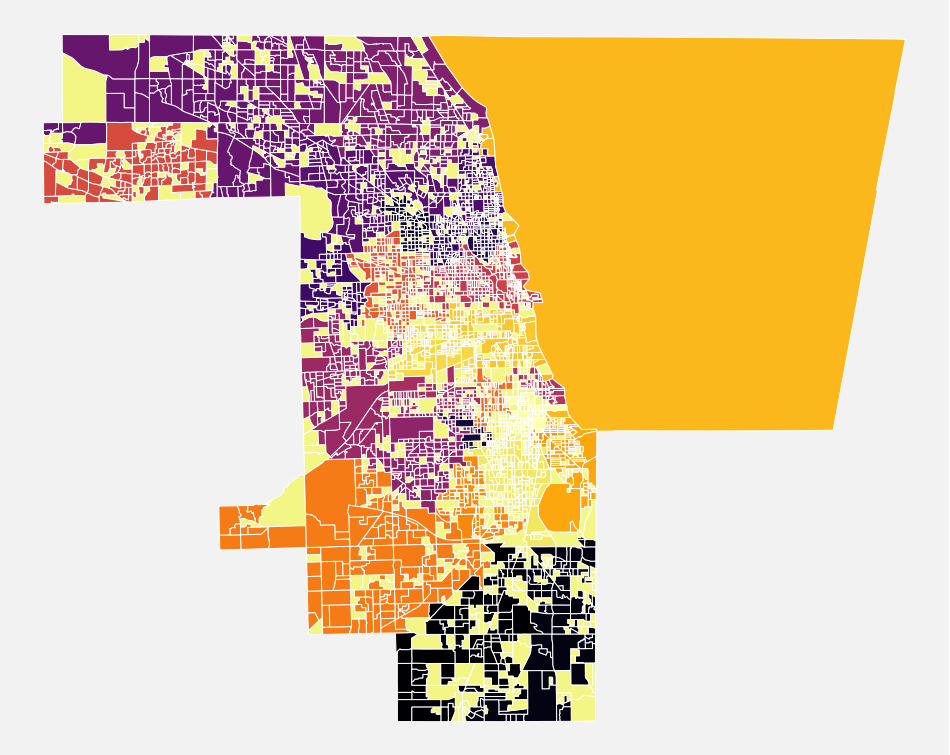
```
p_moran
```




```
p_louvain
```

Louvain Clustering of Predicted Socioeconomic Transitions

Spatial communities of similar socioeconomic change (2015–2025)



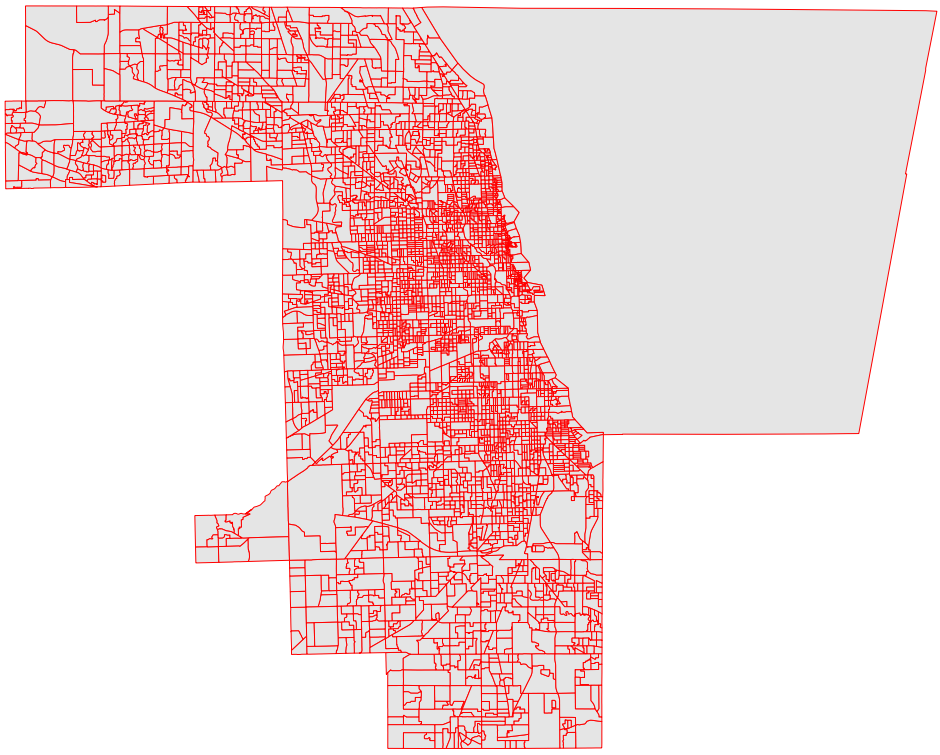

```
cat("STEP 7 COMPLETED SUCCESSFULLY\n")
```

```
## STEP 7 COMPLETED SUCCESSFULLY
```

Plots used for Poster Presentation :

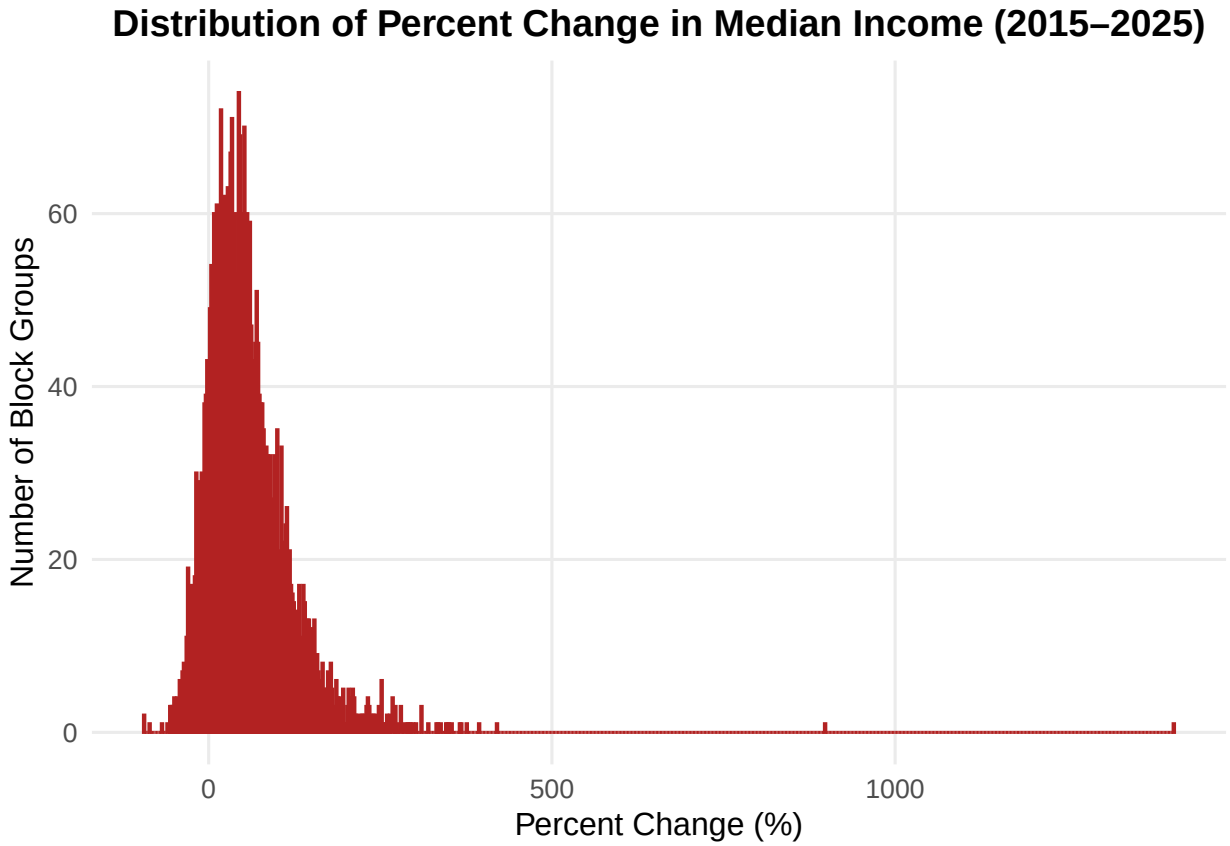
```
library(tigris)
library(sf)
library(ggplot2)

# Load Chicago block groups
bg_chicago <- block_groups(state = "IL", county = "Cook", year = 2021) %>%
  st_transform(3857)

# Simple outline map
ggplot() +
  geom_sf(data = bg_chicago, fill = "gray90", color = "red", size = 0.1) +
  labs(title = "Chicago Block Groups") +
  theme_void()
```

Chicago Block Groups



```
ggsave("chicago_outline_map.png", width = 6, height = 6, dpi = 300)
```

```
library(ggplot2)

# Remove NA and infinite values safely
income_changes <- temporal_features %>%
  filter(!is.na(inc_pct_change), is.finite(inc_pct_change))

# Histogram of percent change in median income
ggplot(income_changes, aes(x = inc_pct_change * 100)) +
  geom_histogram(binwidth = 2, fill = "firebrick", color = "firebrick", alpha = 0.8) +
  labs(
    title = "Distribution of Percent Change in Median Income (2015-2025)",
    x = "Percent Change (%)",
    y = "Number of Block Groups"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5),
    panel.grid.minor = element_blank()
  )
```




```
ggsave("median_income_change_hist.png", width = 7, height = 5, dpi = 300)
```

```
# Example from your Random Forest
rf_cm$table
```

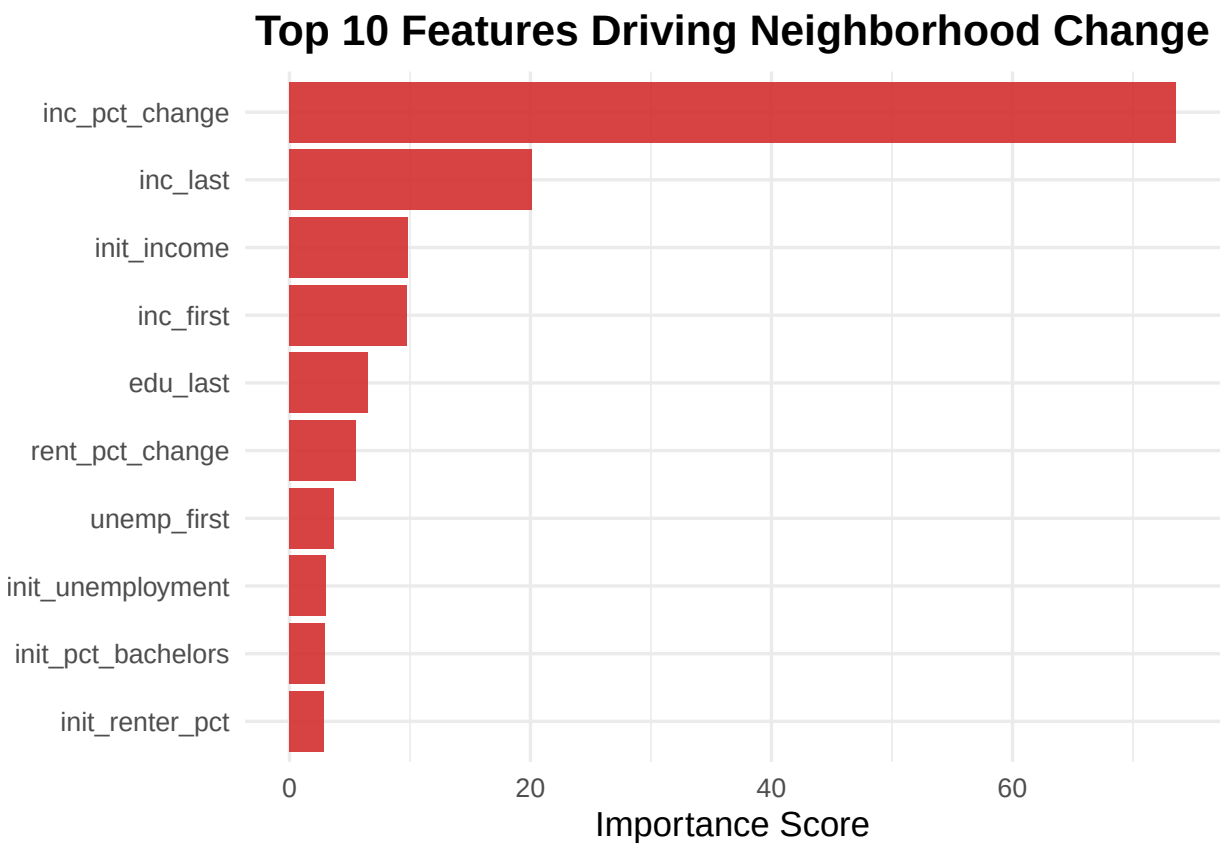
```
##            Reference
## Prediction Decline Stable Growth
##    Decline     107      0      0
##    Stable        0     67      0
##    Growth        0      0    906
```

```
rf_cm$overall["Accuracy"]
```

```
## Accuracy
##        1
```

```
rf_imp_df <- data.frame(
  feature = rownames(rf_imp),
  importance = rf_imp[, 1]
)
library(ggplot2)
top10 <- rf_imp_df %>%
  arrange(desc(importance)) %>%
  slice(1:10)
```

```r
ggplot(top10, aes(x = reorder(feature, importance), y = importance)) +
  geom_bar(stat = "identity", fill = "#D32F2F", alpha = 0.9) +
  coord_flip() +
  labs(
    title = "Top 10 Features Driving Neighborhood Change",
    x = NULL, y = "Importance Score"
  ) +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold", hjust = 0.5))
```




```r
ggsave("Top_10_Features_Driving_Neighborhood_Change.png", width = 7, height = 5, dpi = 300)
```